

Introducción a los conceptos de ingeniería de software

¿Qué es la ingeniería de software?

En 1968, año en el que la Organización del Tratado del Atlántico Norte (OTAN) desarrolló la primera conferencia sobre desarrollo de software, se acuñaron los términos **crisis del software**, para definir los problemas que surgían en el desarrollo de sistemas de software, e **ingeniería del software**, para describir el conjunto de conocimientos que existían en aquel estado inicial.

La crisis del software fue el resultado de proyectos de desarrollo de software que se extendían más allá de los costos y los tiempos planificados (si es que se lograban concluir, ya que a veces se debía cancelar el proyecto). De los sistemas que lograban llegar a completarse, se detectó que tenían un alto costo de mantenimiento, en ocasiones equiparable al costo que había insumido su desarrollo. Esto motivó la necesidad de una disciplina que pudiera aportar mejoras a los procesos, incluyendo métodos, técnicas y herramientas que pudieran mejorar la situación. La ingeniería de software contempla todos los aspectos referentes a la producción de software.

En este momento, Fritz Bayer acuñó la definición original durante una: “Establecimiento y uso de principios de ingeniería para obtener software económico que trabaje de forma eficiente en máquinas reales”.

Otra definición generada por la *Institute of Electrical and Electronics Engineers* (IEEE) en el año 1993, dice que se trata de: “La aplicación de métodos sistemáticos, disciplinados y cuantificables para el desarrollo, operación y mantenimiento de software; esto es, la aplicación de la ingeniería al software”. Esta definición también contempla el estudio de la aplicación de estos métodos.

Una de las definiciones que parece más completas para comprender a qué refiera la disciplina de ingeniería del software fue aportada por un reconocido autor de esta área, Barry W. Boehm, que dice que se trata de “la aplicación práctica del conocimiento científico al diseño y construcción de programas y la documentación asociada requerida para desarrollar, operar y mantenerlos.”

El proceso y el producto de software

¿Qué es un producto de software?

Cuando hablamos de un producto de software es común asociarlo a los programas ejecutables que se desarrollan, aunque esto es erróneo. No solo es el código objeto y el manual del usuario que se entrega al cliente, sino también los documentos asociados producidos durante el proceso mediante los que se desarrolló el software, como pueden ser la documentación de los requerimientos, diseño, el código fuente, los datos de prueba, etc.

Entonces, ¿qué es el proceso?

El proceso es el que se usa para generar un producto de software a partir de los requerimientos del usuario. Ian F. Sommerville define el proceso del software como “un conjunto de actividades cuya meta es el desarrollo o evolución del software”.

El rol del ingeniero del software

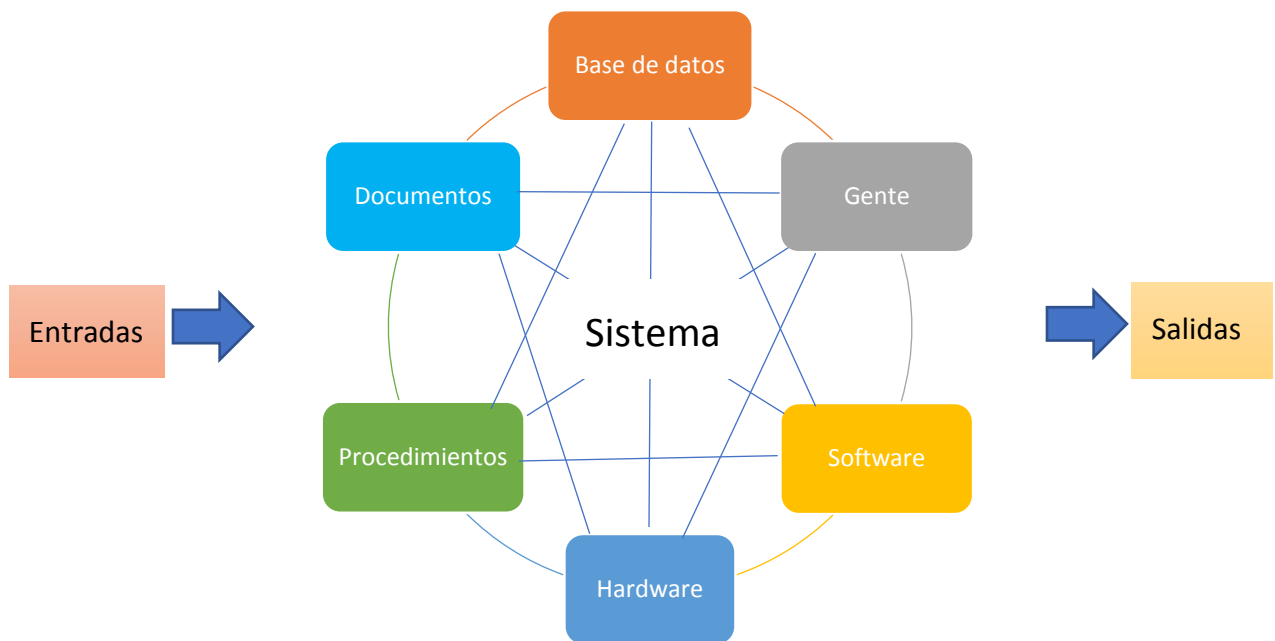
Idealmente, un buen ingeniero de software debe prepararse para enfrentar varios desafíos durante su desarrollo como profesional. Para poder hacerlo debería:

- poseer experiencia en programación;
- conocer de estructuras de datos,
- contar con capacidad para poder traducir requerimientos y deseos vagos donde debería haber una especificación clara y precisa de requerimientos;
- poder manejar diferente vocabulario de forma tal de poder usar conceptos técnicos con el equipo de desarrollo, pero hacerlo en términos de la aplicación al momento de interactuar con un usuario del sistema;
- contar con habilidad para moverse en diferentes niveles de abstracción en las diferentes etapas del proyecto;
- conocer acerca de modelado de procesos y de datos para poder construirlos, usarlos y comprenderlos
- tener habilidad para la comunicación interpersonal y la administración del trabajo, ya que se trata de una disciplina que no se lleva a cabo en forma individual sino en interacción con otros.

Sistemas vs. software

¿Qué es un sistema?

La definición de sistemas indica que se trata de una “colección de componentes organizados para cumplir una función o conjunto de funciones específicas”. IEEE Standard 610.12-1990. Los elementos componentes de un sistema basado en computadoras involucran no solo los programas de computadora (software), los dispositivos electrónicos necesarios (hardware), la gente (usuarios y quienes operan el sistema), la base de datos (donde está la información que maneja el software), la documentación (manuales para operar) y los procedimientos (indican el uso de cada elemento del sistema).



¿Qué es el software?

Como se puede observar en el gráfico anterior, el software es un elemento componente de un sistema que, a diferencia del hardware (físico), es lógico. Es un sistema, o sub-sistema, formado por una colección de programas y documentación que de forma conjunta satisfacen unos determinados requisitos.

Un sistema de software puede ser en sí mismo un sistema independiente que, por ejemplo, realiza su objetivo en un ordenador independiente. También puede formar parte de un sistema mayor, en cuyo caso se trata en realidad de un sub-sistema de software.

Por ejemplo, el sistema de GPS del celular es, en realidad, un sub-sistema del software del celular.

Definición de calidad del software y vistas

La calidad siempre ha sido un objetivo fundamental de las empresas, y es un término que con frecuencia se interpreta mal cuando se utiliza dentro del contexto del software. Calidad implica excelencia de algún tipo, pero por desgracia este no es el significado que le quieren dar los profesionales en la ingeniería de software.

El *American Heritage Dictionary* define la calidad como “una característica o atributo de algo”. Como un atributo de un elemento, la calidad se refiere a las características mensurables, cosas que se pueden comparar con estándares conocidos como longitud, color, propiedades eléctricas, etc. Sin embargo, el software, en su gran extensión como entidad intelectual, es más difícil de caracterizar que los objetos físicos. Según la definición de calidad de la IEEE: “Es cuando el software posee una buena combinación de atributos deseados y el cliente o usuario percibe que el producto cumple con sus expectativas.”

Si les preguntamos a los diferentes jugadores y los involucrados en el sistema, veremos que cada uno tiene una vista diferente de qué es para ellos la calidad. Así, por ejemplo, para el usuario podría ser que su producto de software sea confiable, eficiente y fácil de usar, mientras que para el desarrollador puede ser que dicho producto pueda ser fácil de verificar o de mantener. Por otro lado, el administrador del proyecto buscará atributos como un buen nivel de productividad y eficiencia en el proceso de desarrollo.

La calidad en el software depende del producto que estamos construyendo y del grado en que él posee de la combinación deseada de atributos. Uno de los objetivos primordiales de la ingeniería de software es construir productos de calidad.

Así, podemos concluir que la calidad es la totalidad de características de un producto o servicio que se refieren a su habilidad para satisfacer necesidades establecidas o implicadas.

Requerimientos de calidad del software, el producto y el proceso.

La gestión de calidad propone que la calidad del proceso de desarrollo afecta en forma directa a la calidad de los productos resultantes del proceso. Por ejemplo, en la

producción de hardware la calidad se obtiene por un buen diseño, pero luego en la fase correspondiente a la producción no suelen presentarse problemas de calidad o, simplemente, se monitorea para corregir cualquier desvío. Hay un claro vínculo entre la calidad del proceso y la calidad del producto.

Por otro lado, el software no se fabrica; no se puede gestionar su producción de la misma manera. El desarrollo de software es un proceso más creativo que mecánico, donde la experiencia y habilidades individuales son importantes, y la calidad del producto se verá afectada por factores externos.

En el desarrollo software, por lo tanto, la relación entre la calidad del proceso y la calidad del producto es mucho más compleja. A pesar de esto, la experiencia demuestra que la calidad del proceso tiene una influencia significativa en la calidad del producto de software que se genera, minimizando los defectos en el resultado final. Por esto insistiremos con el aspecto de la calidad.

Cualidades del producto y del proceso

El producto de software es diferente a otros productos y una de las características que lo diferencia es su maleabilidad: puede ser modificado con facilidad. Así, si el producto se está empleando con éxito en la organización, cada nuevo cambio que se requiera (alguna modificación en el entorno o en la forma de operar) se esperará que el producto de software se modifique y adapte al mismo.

Las cualidades del software son características que se presentan en el producto de software o en su proceso de desarrollo.

Según Ghezzi, pueden clasificarse en:

Cualidades internas: si son relevantes para los desarrolladores.

Cualidades externas: si son visibles o relevantes para los usuarios.

Veremos cuáles son las cualidades a tener en cuenta como atributos de calidad:

Correctitud: un producto es “funcionalmente correcto” si se comporta de acuerdo a la especificación de funciones que debería proveer. Es común escuchar el término “correcto” más que “funcionalmente correcto”.

Confiabilidad: informalmente, el software es confiable si el usuario puede depender de él. Formalmente, se define en términos de comportamiento estadístico como la probabilidad

de que el software opere como se esperaba dentro de un intervalo de tiempo especificado.

Robustez: un producto es robusto si se comporta razonablemente, aun en circunstancias que no fueron anticipadas en la especificación de requerimientos. Por ejemplo, que no se produzca un error irrecuperable tan pronto como el usuario presiona una tecla errónea.

Performance: eficiencia del producto de software refiere a que utilice los recursos en forma económica, como por ejemplo tiempo o memoria.

Amigabilidad: un producto de software se dice que es amigable cuando el usuario lo encuentra fácil de usar. La amigabilidad es más que la interfaz de usuario, aunque esta es un componente importante. Las cualidades de correctitud y performance también afectan la amigabilidad.

Si se producen respuestas erróneas no es amistoso, aunque su interfaz lo sea; por otro lado, si las respuestas son demasiado lentas, no importa que posea lindos gráficos o los colores de las pantallas.

Verificabilidad: refiera a la facilidad de prueba del producto. un producto es verificable si sus propiedades pueden ser fácilmente comprobadas. El diseño modular, prácticas de codificación disciplinadas y el uso de un lenguaje de programación apropiado contribuyen a la verificación.

Mantenibilidad: Este término es usado comúnmente para referirse a las modificaciones que son hechas a un producto de software después de la entrega inicial. El mantenimiento se refiere en general a la etapa de “arreglar errores”, aunque también puede incluir el incorporar mejoras ante nuevos requerimientos o cambios como, por ejemplo, modificaciones de dispositivos de hardware o nuevas normativas a las que el software debe adaptarse.

Reusabilidad: se suele aplicar más a una parte de la aplicación que al producto entero. La reusabilidad es difícil de lograr *a posteriori*, por eso se debe pensar en ella cuando los componentes de software son desarrollados, lo que a veces significa invertir más tiempo.

Portabilidad: el software es portable si puede “correr” en distintos ambientes, considerando ambiente como una plataforma de hardware, un ambiente de software o un sistema operativo.

Interoperabilidad: se refiere a la habilidad de un sistema para coexistir y cooperar con otros sistemas. Por ejemplo, que permita importar y exportar datos de y a otros productos. Para esto, un concepto importante es la estandarización de interfaces.

Productividad: mide la eficiencia del proceso de producción de software a través de la cantidad de recursos requeridos. Un proceso eficiente resulta en una entrega más rápida del producto. En general, incluye herramientas, procesos y código necesarios para ejecutar una función. La incorporación de rutinas reusables influye y mejora la productividad.

Oportunidad: es una cualidad relacionada al proceso y se refiere a la habilidad de entregar un producto a tiempo. Requiere de una programación cuidadosa de los tiempos y recursos, buena estimación del trabajo y metas claramente definidas.

Visibilidad: un proceso de desarrollo es visible si todos sus pasos y su estado actual son documentados claramente. La idea es que todos los pasos y el estado del proyecto estén disponibles y fácilmente accesibles para examinación externa.

Integridad: es un atributo que controla los accesos no autorizados a datos o software.

*Hemos clarificados los conceptos de varias cualidades, pero es importante destacar que no todas ellas le serán pedidas a cada producto o proceso de desarrollo que llevemos a cabo, sino que simplemente se deben identificar cuales son relevantes a la hora de establecer los requisitos a cumplir. Por ejemplo, no todos los productos de software que se desarrollan son portables, ya que esto tiene que ver con quién o quiénes serán sus clientes para determinar si es necesario que el nuevo producto cumpla con esta cualidad.

Principios de la ingeniería de software

A diferencia de las cualidades, los principios se aplican a través del proceso de desarrollo y durante la evolución del software. Al ser de aplicabilidad general, constituyen la piedra angular de la ingeniería del software, más que de una fase aislada del ciclo de vida.

Veamos cuales son dichos principios:

Rigor y formalidad: el rigor es el complemento necesario a la creatividad en cada actividad ingenieril, ya que permite desarrollar productos más confiables y controlar sus costos, a la vez que aumenta la credibilidad de los resultados.

Si bien es una cualidad intuitiva, se puede aplicar en diferentes niveles: el nivel más alto es la formalidad, que requiere que el proceso sea conducido y evaluado mediante el empleo

de leyes matemáticas (retomaremos este tema en mayor detalle en el último módulo de la asignatura).

Separación de incumbencias o responsabilidades: permite tratar con diferentes aspectos individuales de un problema y de esta manera poder concentrarse sobre cada una separadamente. Así, por ejemplo, en el desarrollo de un sistema de alta magnitud, seguramente tendremos diferentes equipos interactuando (desarrollo, *testing* & QA, redes, etc.).

Modularidad: un sistema complejo puede ser dividido en partes más sencillas llamadas módulos. Esto ayudará no solo a reducir la complejidad, mejorando el entendimiento, sino que también facilitará la modificación o la corrección de algún proceso, puesto que se debe localizar específicamente el componente que se ocupa de esto.

Para que el diseño de los módulos sea considerado de calidad debe cumplir con dos factores: los módulos deben tener **alta cohesión** y **bajo acoplamiento**. La cohesión es la medida de interrelación entre los componentes internos del módulo, mientras que el acoplamiento mide la relación o comunicación entre módulos.

Abstracción: es el proceso por el cual se identifican los aspectos importantes de un fenómeno y se ignoran aquellos que no son relevantes para su comprensión.

Anticipación al cambio: proceso que identifica las necesidades de reparación o evolución de una aplicación a medida que surgen nuevos requerimientos o se modifican los existentes. Se pueden emplear herramientas para el control de versiones y revisiones del producto para mantenerlo de una forma controlada. De esto se ocupa la Administración de configuraciones.

Generalización: descubrir un problema más general oculto atrás del que se trata.

Incrementalidad: se puede aplicar identificando los subconjuntos de una aplicación que deba ser desarrollada de forma tal de realizarle entregas al usuario y obtener de él una pronta realimentación. Los productos intermedios pueden ser prototipos del final.

*Dentro de lo que son las buenas prácticas de esta disciplina, es recomendable tener siempre presente estos principios y fomentar su aplicabilidad.

Esencia y accidente

Como indica Frederick Brooks en su artículo “No silver bullet”, todo proceso de construcción de un producto de software involucra ciertas dificultades que pueden clasificarse en **esenciales** y **accidentales**.

Citando a Aristóteles, las dificultades esenciales son inherentes a la naturaleza del software, como ser: complejidad, conformidad, variabilidad o mutabilidad y la invisibilidad, factores que pese al progreso tecnológico siguen presentes.

Por otro lado, las dificultades accidentales son las que se encuentran en el momento, que asisten a la producción pero que no son inherentes, por lo que con los avances tecnológicos pueden ir variando (por ejemplo, lenguajes de programación, limitaciones en espacio o velocidad de los dispositivos que se emplearán).